



Embedded Systems & Microcontrollers

Unit 2

RV University | CIE 1 Preparation | 2nd Semester CSE

1. Microprocessors vs Microcontrollers (μ P vs μ C)

1.1 Core Distinction

Attribute	Microprocessor (μ P)	Microcontroller (μ C)
Definition	CPU only; all peripherals external	CPU + Memory + Peripherals on a single chip
Memory	External RAM, ROM, Flash	On-chip SRAM, EEPROM, Flash
Peripheral Integration	External (separate ICs required)	Internal (UART, ADC, Timers, I ² C, SPI)
Pin Count (typical)	~423 (e.g., Intel Pentium 4)	~32 (e.g., ATmega328)
System Complexity	Requires many chips; complex PCB design	Single-chip solution
Power Consumption	Higher (external bus driving, more logic)	Lower (internal bus, simpler core)
Cost per unit	Higher for complete system	Lower for embedded tasks
Use Cases	General-purpose computing (PCs, servers)	Embedded, real-time control
Instruction Set	Complex (CISC), large ISA	Simple (RISC), compact ISA

1.2 Why μ P Memory is External

The microprocessor core is architecturally complex (out-of-order execution, large caches, superscalar pipelines). This complexity consumes almost the entire die area, leaving no physical space for on-chip memory. Therefore:

- **L1 cache** is on-die (tiny, fast)
- **L2/L3 cache, RAM, Flash** are off-chip

1.3 Why μ C Memory is Internal

MCU cores use **simpler RISC architectures** (fewer transistors, smaller die area), leaving ample silicon real estate to integrate:

- **SRAM** (data memory, volatile)
- **Flash** (program memory, non-volatile)
- **EEPROM** (non-volatile byte-erasable storage)
- **Peripherals** (UART, SPI, I²C, ADC, Timers, etc.)

Only **I/O pins** protrude out of the chip package.

2. Different Levels of Implementation

Three distinct tiers exist based on integration level:

Tier 1 — Microprocessor Level

- **On-chip:** CPU core + L1 cache only
- **Off-chip:** L2 cache, RAM, peripheral ICs
- Example: Intel Pentium, AMD Ryzen

Tier 2 — SoC (System-on-Chip) / SSD Controller Level

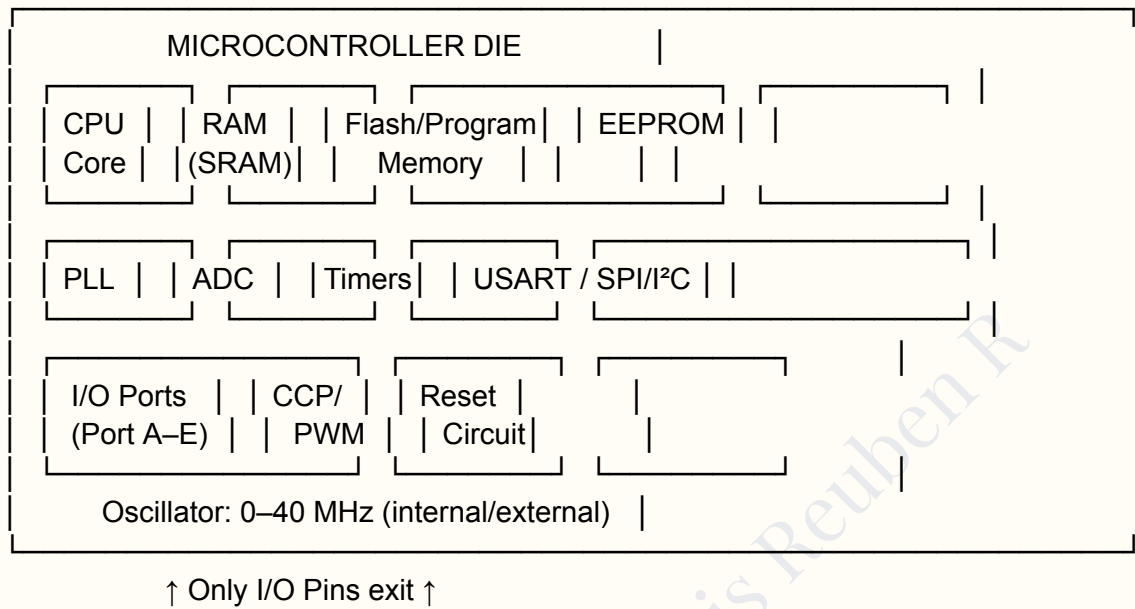
- **On-chip:** CPU core + all cache levels
- **Off-chip:** RAM (DRAM), Flash storage
- Example: SSD controllers, smartphone SoCs
- **SATA** = Serial ATA (Serial Advanced Technology Attachment) — hard disk interface
- **SSD** = Solid State Device using Flash memory

Tier 3 — Microcontroller Level (*highest integration*)

- **On-chip:** CPU core + SRAM + EEPROM + Flash + ALL peripherals
 - **Off-chip:** Nothing except I/O pin connections
 - Example: ATmega328 (Arduino), RP2040 (Raspberry Pi Pico)
-

3. Inside a Microcontroller

A microcontroller integrates all of the following subsystems on a single die:



Key Internal Modules Explained

Module	Full Form	Function
USART	Universal Synchronous/Asynchronous Receiver-Transmitter	Serial communication; can operate in both sync and async modes
EEPROM	Electrically Erasable Programmable Read-Only Memory	Byte-level non-volatile storage; survives power cycles
ADC	Analog-to-Digital Converter	Converts real-world analog signals to digital values
PLL	Phase-Locked Loop	Clock multiplier; generates higher internal frequencies from a lower crystal reference
CCP/PWM	Capture/Compare/PWM	Generates Pulse-Width Modulated signals; used for motor control, LED dimming
Oscillator	—	Clock source; 0–40 MHz range (crystal or internal RC oscillator)

4. Why MCUs Consume Less Power

All four reasons are valid simultaneously (Quiz 1 Answer: A, B, C, D)

Reason	Explanation
A — Simpler core	Fewer transistors switching per cycle → lower dynamic power
B — On-chip memory	No need to drive off-chip memory bus (eliminates capacitive bus loading)
C — On-chip peripherals	Peripheral access stays on-chip; no long PCB trace driving
D — Internal bus accesses	Internal buses are short, low-capacitance; driving external buses requires significantly more current

Root cause: Power = $C \cdot V^2 \cdot f$ (capacitance × voltage² × frequency). External buses have large parasitic capacitance → high power. Internal buses have minimal capacitance → low power.

5. Other Popular MCU Families

Family	Manufacturer	Architecture	Bit Width	Notes
8051	Intel (originally)	CISC	8-bit	Historic; still widely taught
ARM7, ARM9, ARM11	ARM Holdings	RISC	32-bit	Used in embedded Linux systems
MSP430	Texas Instruments	RISC	16-bit	Ultra-low power; popular in IoT
PIC	Microchip Technology	RISC	8–32-bit	Wide range of variants
Renesas RX/RL78	Renesas Electronics, Tokyo	RISC	8–32-bit	Industrial-grade MCUs
STM32	STMicroelectronics, Switzerland	ARM Cortex-M	32-bit	Very popular in hobbyist+industry
RP2040	Raspberry Pi Ltd, UK	ARM Cortex-M0+	32-bit	Dual-core; this course's primary MCU

6. Raspberry Pi Family — Introduction

6.1 Foundation Background

- **Organization:** Raspberry Pi Foundation — a British registered charity (not-for-profit)
- **Founded:** 2009
- **Mission:** Promote the study of basic computer science in schools
- **Products:** Raspberry Pi single-board computers (SBCs) and microcontrollers

6.2 Board Comparison Table

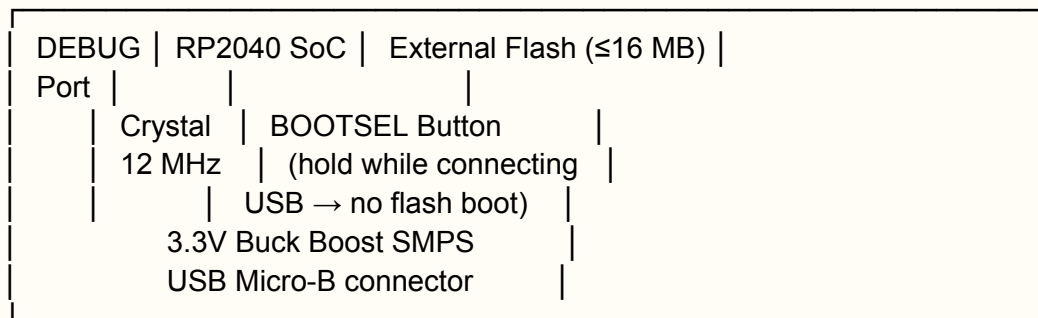
Board	Release	Architecture	Category	Notes
Raspberry Pi Zero	Nov 2015	ARM11 (BCM2835)	SBC	Minimal, credit-card sized
Raspberry Pi 3 Model B+	2016	ARM Cortex-A53 (BCM2837B0)	SBC	Quad-core 1.4 GHz, WiFi/BT
Raspberry Pi 4 Model B	Jun 2019	ARM Cortex-A72 (BCM2711)	SBC	Up to 8 GB RAM, USB 3.0
Raspberry Pi Pico	Jan 2021	ARM Cortex-M0+ (RP2040)	MCU Board	\$4, no WiFi
Raspberry Pi Pico W	Jan 2022	ARM Cortex-M0+ (RP2040)	MCU Board	\$6, adds 2.4 GHz WiFi

Critical distinction: SBC boards (Pi 3, Pi 4) run full Linux OS. **Pico/Pico W are bare-metal microcontroller boards** — no OS, direct hardware programming.

6.3 Raspberry Pi Pico — Key Characteristics

- **First board** based on a Raspberry Pi-designed silicon chip (RP2040) — chip designed in the UK
- Programmable in: **MicroPython, CircuitPython, C/C++**
- Designed for **physical computing** (GPIO, sensors, actuators) — analogous in purpose to Arduino
- **Retail price:** \$4.00 USD

6.4 Raspberry Pi Pico Layout



- **SoC:** System on a Chip (RP2040)
- **SMPS:** Switched Mode Power Supply (efficient DC-DC conversion)
- **BOOTSEL button:** When held during USB plug-in, board appears as USB mass storage → enables drag-and-drop programming (UF2 format)
- **Major limitation of Pico:** No built-in WiFi module

6.5 Raspberry Pi Pico W — Added Features

- Released **January 2022** at **\$6**
- Adds **2.4 GHz 802.11b/g/n Wi-Fi chipset + on-board antenna**
- Form factor **unchanged** — same dimensions, same pin layout as original Pico
- **Pin-compatible** with Pico → drop-in replacement in existing projects
- Software/firmware support largely unchanged

7. RP2040 — Introduction & Physical Details

7.1 Physical Package

- **Package type:** QFN-56 (Quad-Flat No-leads, 56 pins)
 - **QFN** = Quad-Flat No-leads — a Surface-Mount Device (SMD) package with pads on the underside, not protruding leads
- **Mounting:** Surface Mount Technology (SMT)
- **Dimensions:** 7 mm × 7 mm × 0.9 mm
- **Country of Origin:** TW (Taiwan)
- **Pin count:** 56 (visible on underside pads)

7.2 Full Specification Table

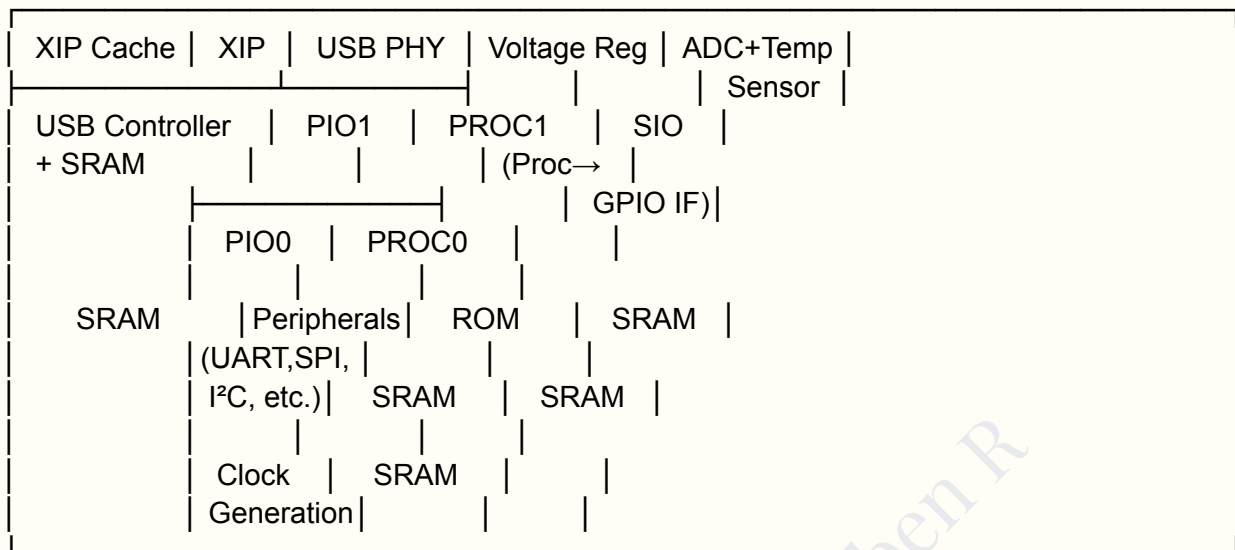
Parameter	Value
Brand	Raspberry Pi
Family Name	ARM
Package Type	QFN (Quad-Flat No-leads)
Mounting Type	Surface Mount
Pin Count	56
Device Core	ARM Cortex-M0+
Data Bus Width	32-bit
Program Memory Size	16 MB (external Flash via QSPI)
Maximum Frequency	133 MHz
RAM Size	264 kB (on-chip SRAM)
USB Channels	1 × Device, 1 × Host
Number of PWM Units	2 × 32-bit (16 channels total)
Number of SPI Channels	2
Typical Operating Supply Voltage	3.63 V
Height	0.9 mm
Width	7.75 mm
Length	7.75 mm
Dimensions	7 × 7 × 0.9 mm
Minimum Operating Temperature	−20 °C
Maximum Operating Temperature	+85 °C
Instruction Set Architecture	RISC
Program Memory Type	Flash (NOR Flash)
ADCs	4 × 12-bit

Number of I ² C Channels	2
Number of UART Channels	2
Number of ADC Units	1 (4-channel multiplexed)
Number of Timers	4

7.3 RP2040 Pin Descriptions

Pin / Signal	Description
GPIOx	General-purpose digital input/output
GPIOx/ADCy	GPIO pin with additional analog-to-digital converter function
QSPIx	Interface to SPI, Dual-SPI, or Quad-SPI Flash; supports Execute-in-Place (XIP)
USB_DM / USB_DP	USB controller; full-speed device and full/low-speed host
XIN / XOUT	Crystal oscillator connection to RP2040's internal oscillator circuit
RUN	Global asynchronous reset: driven LOW → reset; driven HIGH → run
SWCLK / SWDIO	Serial Wire Debug (SWD) multi-drop bus; debug access to both processors
TESTEN	Factory test mode pin (leave unconnected in normal use)
GND	Single external ground; internally bonded to multiple ground pads on die
IOVDD	Power supply for digital GPIOs: nominal 1.8 V – 3.3 V
USB_VDD	Power supply for USB PHY: nominal 3.3 V
ADC_AVDD	Power supply for ADC: nominal 3.3 V
VREG_VIN	Power input for internal core voltage regulator: 1.8 V – 3.3 V
VREG_VOUT	Power output from internal core regulator: nominal 1.1 V, max 100 mA
DVDD	Digital core power supply: nominal 1.1 V

7.4 Die Layout (Internal Floor Plan)



8. RP2040 — System Overview & Key Features

8.1 System Overview Diagram — Module Explanations

Acronym	Full Form	Role in RP2040
PLL	Phase-Locked Loop	Two on-chip PLLs: one for USB clock, one for core clock
SIO	Single-Cycle I/O	Direct processor-to-GPIO interface; zero-latency register access
SWD	Serial Wire Debug	2-wire debug protocol for both processors
GPIO [29:0]	General-Purpose I/O	30 programmable I/O pins
SPI	Serial Peripheral Interface	High-speed synchronous serial communication
QSPI	Quad SPI	4-bit wide SPI for external Flash; supports XIP
XIP	Execute In Place	Fetch and execute code directly from external Flash without copying to RAM
SRAM	Static RAM	264 kB total; 6 independent banks for concurrent access

DMA	Direct Memory Access	Data transfers between peripherals/memory without CPU involvement
UART	Universal Async Receiver/Transmitter	Asynchronous serial communication (RS-232 compatible)
PWM	Pulse Width Modulator	Generates variable duty-cycle digital signals
RTC	Real-Time Clock	Maintains current time/date independently
I²C	Inter-Integrated Circuit	2-wire synchronous serial bus for slow peripherals
ADC & TS	ADC + Temperature Sensor	4-channel 12-bit ADC; built-in temperature sensor on channel 4
PIO	Programmable I/O	2 PIO blocks × 4 state machines = 8 state machines; implement custom protocols in hardware
Watchdog	—	Hardware timer that resets the chip if not periodically "kicked" by software
Sysctrl	System Control	Controls clock distribution, resets
Sysinfo	System Information	Read-only chip identification registers

8.2 Key Features Summary

CPU: Dual ARM Cortex-M0+ @ up to 133 MHz

Memory: 264 kB on-chip SRAM (6 banks)

Up to 16 MB off-chip Flash via QSPI bus (XIP capable)

Architecture: DMA controller

Fully connected AHB crossbar switch

Interpolator and integer divider peripherals

On-chip programmable LDO (core voltage generation)

Two on-chip PLLs (USB clock + core clock)

Interfacing: 30 GPIO pins (4 usable as ADC analog inputs)

Peripherals: 2 × UARTs

2 × SPI controllers

2 × I²C controllers

16 × PWM channels

1 × USB 1.1 (host + device)

8 × PIO state machines

Package: 7 × 7 mm QFN-56

- **AHB** = Advanced High-performance Bus — ARM's bus standard for high-bandwidth on-chip interconnect
- **LDO** = Linear Low-Dropout Regulator — generates the 1.1 V digital core voltage from VREG_VIN

9. RP2040 — ROM/RAM Details

9.1 Memory Map Summary

Memory Region	Size	Type	Notes
ROM	16 kB	Read-Only	Contains bootloader, USB mass-storage code, math libraries
Internal SRAM	264 kB	SRAM (volatile)	$4 \times 64 \text{ kB} + 2 \times 4 \text{ kB} = 264 \text{ kB}$ total
XIP/Cache	16 kB	SRAM (repurposable)	Used as instruction cache for XIP; can be freed if XIP disabled
USB DPRAM	4 kB	SRAM (repurposable)	Used by USB controller; available as RAM if USB is unused
External Flash	Up to 16 MB	NOR Flash (external)	Via QSPI; not part of RP2040 SoC die

9.2 SRAM Bank Breakdown

6 independent SRAM banks:

Bank 0: 64 kB
 Bank 1: 64 kB
 Bank 2: 64 kB
 Bank 3: 64 kB
 Bank 4: 4 kB
 Bank 5: 4 kB
 TOTAL: 264 kB

→ 256 kB (main data/code memory)

→ 8 kB (scratch/stack memory)

Why 6 independent banks? Each processor (PROC0, PROC1) and the DMA can access different banks simultaneously without bus conflicts → true parallel memory access.

9.3 Maximum Usable RAM Calculation

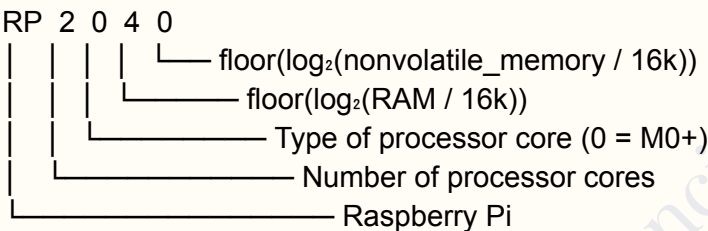
Base SRAM: 264 kB
+ XIP/Cache (if XIP off): 16 kB
+ USB DPRAM (if USB off): 4 kB

Maximum total RAM: 284 kB

Note: 16 MB of Flash is **external** to the RP2040 SoC — it connects via QSPI bus on the Pico board. It is **not** part of the RP2040 die.

10. RP2040 — Nomenclature Decoding

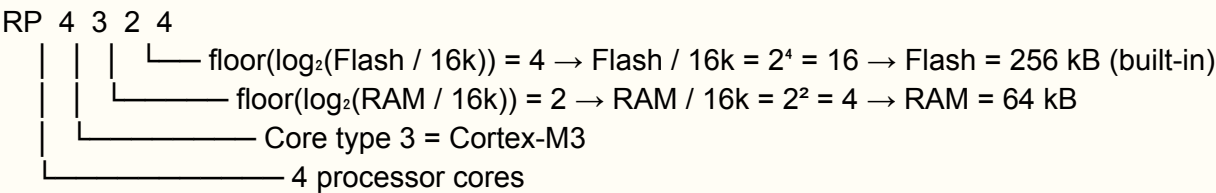
10.1 Naming Convention



10.2 RP2040 Decoded

Field	Value	Calculation
Brand	RP = Raspberry Pi	—
Cores	2	Dual-core
Core Type	0 = Cortex-M0+	ARM naming
RAM digit	4	floor(log ₂ (264k / 16k)) = floor(log ₂ (16.5)) = floor(4.04) = 4
Flash digit	0	No on-chip nonvolatile storage → 0

10.3 Practice Quiz: RP4324 Decoded



Answer: 4-core Cortex-M3 chip with 64 kB RAM and 256 kB built-in Flash.

11. RP2040 — Feature Summary

Complete feature list for exam quick reference:

- ☒ Dual-core ARM Cortex-M0+ @ 133 MHz
- ☒ 264 kB on-chip SRAM
- ☒ Up to 16 MB off-chip Flash via QSPI (XIP supported)
- ☒ DMA controller (frees CPU from data-move tasks)
- ☒ 30 GPIO pins; 4 usable as ADC analog inputs (GP26–GP29)
- ☒ 2 × UARTs, 2 × SPI, 2 × I²C controllers
- ☒ 8 × PIO state machines (2 blocks × 4)
- ☒ Accurate clock and timer on-chip
- ☒ On-chip temperature sensor (ADC channel 4)
- ☒ 16 × PWM channels
- ☒ 1 × USB 1.1 with host and device support
- ☒ Accelerated floating-point libraries in ROM
- ☒ USB mass-storage boot mode (UF2 drag-and-drop programming)
- ☒ Product lifetime guaranteed until at least **January 2041**

12. Basic Concept of Pipelining

12.1 Definition

Pipelining is a technique of decomposing a sequential process into sub-operations, with each sub-process executed in a dedicated segment that operates **concurrently** with all other segments.

Analogous to an assembly line: multiple instructions are at different stages of completion simultaneously.

12.2 Standard 5-Stage RISC Pipeline

FETCH → DECODE → EXECUTE → MEMORY → WRITE BACK
(IF) (ID) (EX) (MA) (WB)

Stage	Abbreviation	Function
Instruction Fetch	IF	Read next instruction from memory; update PC
Instruction Decode	ID	Decode opcode; identify operation type, registers, immediate values

Execute	EX	ALU performs computation (arithmetic, logic, address calculation)
Memory Access	MA	Load data from / store data to memory (only for LDR/STR instructions)
Write Back	WB	Write result back into destination register

12.3 Sequential (Unpipelined) vs Pipelined Execution

Sequential: Each instruction completes all 5 stages before the next begins.

Cycle: 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
Instruction 1: IF ID EX MA WB
Instruction 2: IF ID EX MA WB
Instruction 3: IF ID EX MA WB

→ 3 instructions take **15 cycles**

Pipelined: Stages overlap.

Cycle: 1 2 3 4 5 6 7
Instruction 1: IF ID EX MA WB
Instruction 2: IF ID EX MA WB
Instruction 3: IF ID EX MA WB

→ 3 instructions complete in **7 cycles** (CPI approaches 1 at steady state)

12.4 RISC Load/Store Architecture

- **Key constraint:** Memory access stage (MA) is only required for **load (LDR)** and **store (STR)** instructions.
- Arithmetic/logical instructions operate **register-to-register** → MA stage is a NOP for them.
- This simplifies pipeline design and allows uniform stage timing.

12.5 Pipeline Depth Trade-offs

Pipeline Depth	Examples	Characteristic
3–6 stages	ARM Cortex-M0+, M3, M7	Simple, low power, low branch penalty
8–14 stages	ARM Cortex-A7, Intel i3/i5/i7	Higher clock frequency, greater throughput
31 stages	Intel Pentium 4 Cedar Mill	Extreme clock speed; severe branch misprediction penalty

Trade-off principle:

- **More stages** → shorter critical path → higher maximum clock frequency → higher throughput
- **More stages** → greater pipeline fill time (latency) → worse branch misprediction penalty
- Optimal balance depends on workload characteristics

12.6 Pipeline Stages in Detail

FETCH:

- Retrieves next instruction from program memory using PC as address
- Handles: sequential (PC+4) or branch/jump (new PC) address computation
- Uses dedicated adder (not ALU) to increment PC

DECODE:

- Parses instruction encoding to extract:
 - Opcode (operation type)
 - Source register indices
 - Destination register index
 - Immediate values or offsets
 - Control signals for subsequent stages

EXECUTE:

- ALU performs the operation specified by decoder
- Types: arithmetic (ADD, SUB, MUL), logical (AND, OR, XOR), shift, comparison, address calculation

MEMORY ACCESS:

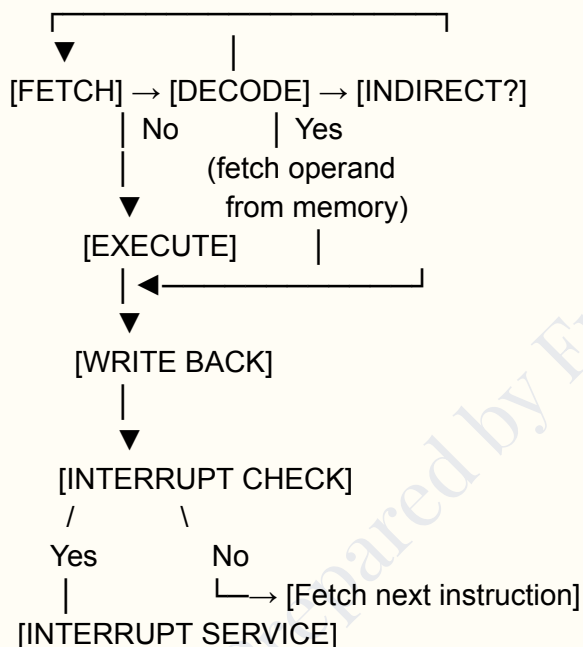
- Active only for load (LDR/LW) and store (STR/SW) instructions
- Reads from or writes to data memory at the computed address

WRITE BACK:

- Writes ALU result or loaded memory value back into destination register
- Completes the instruction

13. Instruction Cycle

13.1 The Instruction Cycle (Von Neumann)



13.2 Each Phase Explained

Phase	Trigger	Action
Fetch	Always	PC → MAR; Memory[MAR] → MBR → IR; PC ← PC + instruction_size
Decode	Always	Decode IR opcode; identify operation, operand types
Indirect	If operand is in memory (indirect addressing)	Compute effective address; initiate memory read cycle

Execute	Always	ALU performs operation on operands
Write Back	If result must be stored	Store result to register or memory
Interrupt	After every instruction completion	Check if interrupt pending; if yes, save CPU state → service ISR

13.3 Addressing Modes

Direct Addressing:

ADD R1, R2, R3 ; $R1 \leftarrow R2 + R3$
 ; All operands are registers — no memory access needed

Indirect Addressing:

ADD R1, R2, [R3+R4] ; $R1 \leftarrow R2 + \text{Memory}[R3+R4]$
 ; [R3+R4] means "contents of memory at address R3+R4"
 ; Requires Indirect cycle to fetch operand from memory

13.4 State Transition Diagram — States

State #	Name	Description
1	Instruction Fetch	Read instruction from program memory
2	Instruction Address Calculation	Compute address of next instruction
3	Source Operand Address Calculation	Compute address of source operand
4	Source Operand Fetch (Indirect)	Fetch source from memory
5	Data Operation	Execute via ALU
6	Destination Operand Address Calculation	Compute write-back address
7	Destination Operand Store (Indirect)	Write result to memory
8	Interrupt Check	Test interrupt pending flag
9	Interrupt Service	Service the interrupt (save context, jump to ISR)
10	Instruction Address Calculation	Prepare next PC value

13.5 Example: ADD R1, R2, [R3+R4] — Full Trace

PC = 100H

State 1 (IF): $MAR \leftarrow 100H$; $IR \leftarrow \text{Memory}[100H] = \{\text{ADD } R1, R2, [R3+R4]\}$

State 2 (IA): $PC \leftarrow 100H + 4 = 104H$

State 2 (ID): Decode: opcode=ADD, dest=R1, src1=R2, src2=Mem[R3+R4]

State 3 (SA): $\text{EffAddr} \leftarrow R3 + R4$

State 4 (SF): $MAR \leftarrow \text{EffAddr}$; $MBR \leftarrow \text{Memory}[\text{EffAddr}] \leftarrow$ Indirect cycle

State 5 (EX): $R1 \leftarrow R2 + MBR$

State 6/7: Dest is register (R1) $\rightarrow$ No memory write needed

State 8 (INT): Check interrupt pending

If interrupt: save PC=104H, jump to ISR

If no interrupt: fetch next instruction at PC=104H

14. Simple CPU Design — General Purpose Register-Based

14.1 Key CPU Registers

Register	Full Name	Role
PC	Program Counter	Holds address of the next instruction to fetch; auto-increments after each fetch; modified by branch/jump instructions
IR	Instruction Register	Holds the currently fetched instruction for decoding
MAR	Memory Address Register	Sole register connected to the address bus; holds address for any memory operation
MBR / MDR	Memory Buffer / Data Register	Bidirectional buffer; holds data being written to or read from memory; directly connected to data bus
SR	Status/Flags Register	Holds condition codes: Zero (Z), Negative (N), Carry (C), Overflow (V)
GPRs	General-Purpose Registers	Fast on-chip storage for operands and results (R0–R15 in ARM)

14.2 PC vs MAR — Distinction

Attribute	Program Counter (PC)	Memory Address Register (MAR)
Scope	Only tracks instruction addresses	Any memory address (instruction or data)
Modification	Auto-incremented each fetch; changed by branches	Loaded from PC (fetch) or computed address (data access)
Direction	Always → memory	→ memory for both reads and writes

14.3 General-Purpose Register Set

- **Created from:** Latches (storage) + Decoder (address selection)
- **Latches:** D-type flip-flops that hold data until written
- **Decoder:** Selects which register to read (source) or write (destination)
- Access speed: Single clock cycle (far faster than SRAM, which may need 2–4 cycles)
- ARM Cortex-M0+ has **16 registers** (R0–R15); R13=SP, R14=LR, R15=PC

15. Instruction Fetch — Detailed Walkthrough

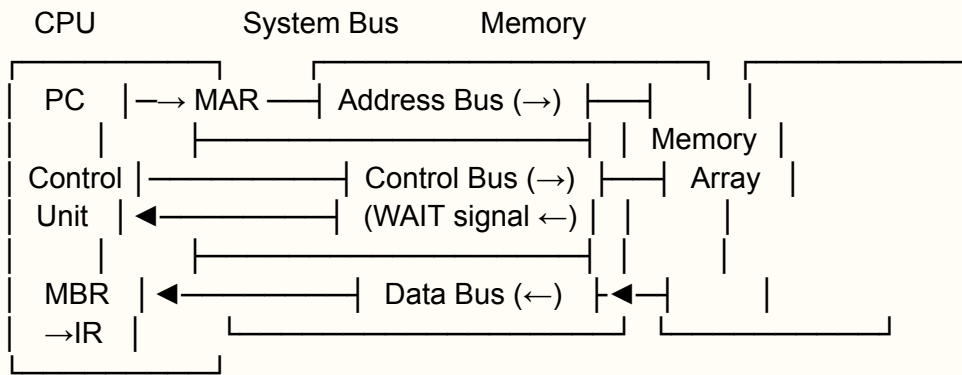
15.1 Three Steps of the Fetch Cycle

Step 1: PC —————→ MAR
(PC contents transferred to MAR)

Step 2: MAR —[address bus]—→ Memory —[data bus]—→ MBR
(Control unit issues READ command on control bus)
(Memory places instruction word on data bus)
(MBR latches the instruction)
SIMULTANEOUSLY: $PC \leftarrow PC + 4$ (prepare for next instruction)

Step 3: MBR —————→ IR
(Instruction now in IR, ready for decode)

15.2 CPU-Memory Interface (Read Cycle)



Control Bus Signals:

- **RD/ $\overline{\text{WR}}$** — Read/Write command from CPU to memory
- **WAIT** — Memory asserts this to tell CPU it needs more time (slow memory)

Bus Directionality:

- **Address bus:** Unidirectional (CPU → Memory always)
- **Data bus:** Bidirectional (CPU→Memory for writes; Memory→CPU for reads)
- **Control bus:** Bidirectional (commands → memory; WAIT ← memory)

15.3 CPU Design Signals During Fetch (Example values)

PC = 10 (binary)

MAR ← 101 (Load MAR signal active)

ALU gets: Operand1 = PC = 10, Operand2 = 00 (constant)

ALU function = ADD (NOP effectively, passes PC value)

Result flows to MAR

Memory addressed at 101 → instruction word read → MBR

16. Instruction Execution — ADD R1, R2, R3

16.1 Instruction Format (32-bit)

31 24 23 16 15 8 7 0



Semantics: $R1 \leftarrow R2 + R3$

16.2 Execution Cycle Data Path

Source Register Select 1 (sl): 010 (R2)
Source Register Select 2 (s2): 011 (R3)
Destination Register Select (d): 001 (R1)

GPR outputs: R_src1 = R2 value, R_src2 = R3 value
MUX Operand1 select: 01 → passes R2
MUX Operand2 select: 01 → passes R3
ALU Function select: ADD
Shifter select: 00 → NOP (no shift)
Result → Destination R1

Load select (Reg Ld): 001 → R1 loaded
Loading pulse writes result into R1

16.3 ADD Signal Summary Table

Signal	Value	Meaning
Source1 select	010	R2
Source2 select	011	R3
Dest select	001	R1
ALU function	ADD	Perform addition
Shifter	NOP (00)	No shift
MAR load	Not active	No memory access
Reg load	Active (001)	Write result to R1

17. Store Instruction — STR R2, [R1]

17.1 Semantics

STR R2, [R1] ; Memory[R1] $\leftarrow$ R2

; Stores the 32-bit value in R2 into the memory address held in R1

This uses **indirect addressing**: [R1] means "the memory location whose address is the value inside register R1."

17.2 Five-Stage Execution Trace

IF (Instruction Fetch):

PC $\rightarrow$ MAR $\rightarrow$ Address Bus $\rightarrow$ Memory

Instruction "STR R2, [R1]" $\rightarrow$ Data Bus $\rightarrow$ MBR $\rightarrow$ IR

PC $\leftarrow$ PC + 4

ID (Decode):

Decode IR: opcode = STR (store), src = R2, address_reg = R1

CPU identifies this as a memory-write operation

EX (Execute):

Effective address = contents of R1 (base register, no offset)

MAR $\leftarrow$ R1 (load MAR with destination memory address)

MA (Memory Access):

MBR $\leftarrow$ R2 (load data register with value to be stored)

MAR $\rightarrow$ Address Bus

MBR $\rightarrow$ Data Bus

Control: assert WRITE command

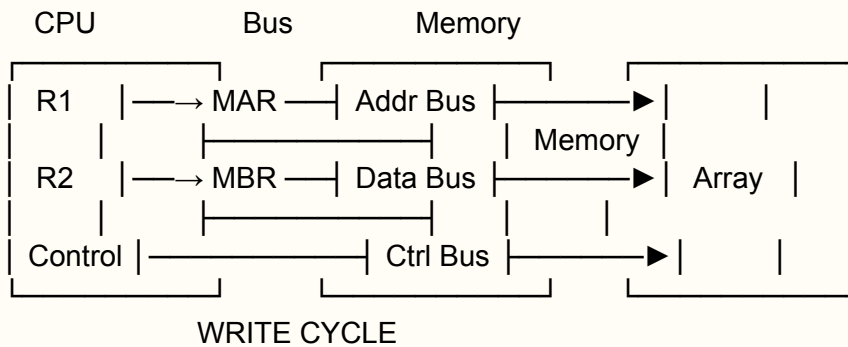
Memory[R1] $\leftarrow$ R2 value (memory write completes)

WB (Write Back):

No register write needed for STR instruction

(Store instructions have no architectural destination register)

17.3 Bus Interaction During STR



17.4 MAR and MBR Roles in STR

Register	Content During STR	Purpose
MAR	Contents of R1 (destination address)	Placed on address bus to select memory location
MBR	Contents of R2 (data to write)	Placed on data bus to supply write data to memory

18. Lab Programs — Series 1

Platform: Raspberry Pi Pico (RP2040) using **MicroPython** in Thonny IDE **GPIO reference:** GP0–GP28 are physical GPIO pins; `machine.Pin` is the MicroPython class

Lab 1.1 — Blink LED_BUILTIN

Objective: Blink the on-board LED (connected to GP25 on standard Pico; on Pico W, handled differently).

Concept: The built-in LED is wired to GPIO 25 on the Raspberry Pi Pico. Setting the pin HIGH turns the LED ON; LOW turns it OFF. Using `time.sleep()` introduces a delay.

Lab 1.1: Blink the built-in LED

The onboard LED is on pin 25 (standard Pico)

```
from machine import Pin # Import Pin class from machine module
import time             # Import time module for sleep functions
```

```
# Configure GP25 as output (the built-in LED pin on Pico)
led = Pin(25, Pin.OUT) # Pin.OUT = output mode
```

```
while True:          # Infinite loop
    led.value(1)      # HIGH → LED ON
    time.sleep(1)     # Wait 1 second
    led.value(0)      # LOW  → LED OFF
    time.sleep(1)     # Wait 1 second
```

Alternative using toggle:

```
from machine import Pin
import time
```

```
led = Pin(25, Pin.OUT)
```

```
while True:
    led.toggle()      # Flip current state (ON→OFF or OFF→ON)
    time.sleep(1)
```

Key API:

Function	Description
<code>Pin(id, mode)</code>	Create pin object; mode: <code>Pin.OUT</code> , <code>Pin.IN</code>
<code>led.value(1)</code> <code>)</code>	Set pin HIGH (3.3 V output)
<code>led.value(0)</code> <code>)</code>	Set pin LOW (0 V output)
<code>led.toggle()</code> <code>)</code>	Invert current output state
<code>time.sleep(n)</code>	Delay n seconds (float supported)

Lab 1.2 — Hello World via Serial Monitor

Objective: Print "Hello World" to the serial monitor (REPL output via USB UART).

Concept: MicroPython's `print()` sends text over the USB serial connection. Thonny's Shell window acts as the serial monitor. `time.sleep()` controls the print rate.

Lab 1.2: Display "Hello World" using the Serial Monitor

```
import time          # Import time module

while True:
    print("Hello World") # Transmit string over USB serial (REPL)
    time.sleep(1)        # Wait 1 second between prints
```

Explanation:

- `print()` in MicroPython on Pico outputs to **stdout**, which maps to the **USB CDC serial port**
- Thonny IDE's Shell panel (or any serial terminal at 115200 baud) displays this output
- No additional import needed — `print` is a built-in

Variant — print with counter:

```
import time

count = 0
while True:
    count += 1
    print(f"Hello World — iteration {count}")
    time.sleep(1)
```

Lab 1.3 — Alternate LED Blink (4 LEDs, 3s and 5s delays)

Objective: Blink 2 groups of 2 LEDs alternately. Group A blinks with 3-second delay; Group B blinks with 5-second delay.

Wiring assumption:

GP0 — 330Ω — LED1(A) — GND
GP1 — 330Ω — LED2(A) — GND
GP2 — 330Ω — LED3(B) — GND
GP3 — 330Ω — LED4(B) — GND

Interpretation: "Alternate" means when Group A is ON, Group B is OFF and vice versa. The delay for Group A ON phase is 3 s; delay for Group B ON phase is 5 s.

```
# Lab 1.3: Blink 2 alternate LEDs out of 4 LEDs
# Group A (LED1, LED2) on GP0, GP1 — delay 3 seconds
# Group B (LED3, LED4) on GP2, GP3 — delay 5 seconds
```

```
from machine import Pin
import time
```

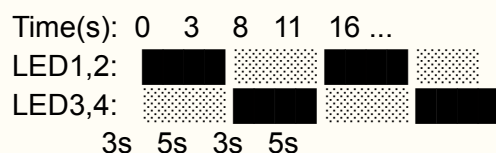
```
# Define pin objects — all as outputs
led1 = Pin(0, Pin.OUT) # Group A
led2 = Pin(1, Pin.OUT) # Group A
led3 = Pin(2, Pin.OUT) # Group B
led4 = Pin(3, Pin.OUT) # Group B
```

```
def all_off():
    """Turn all LEDs OFF"""
    led1.value(0)
    led2.value(0)
    led3.value(0)
    led4.value(0)
```

```
while True:
    # Phase 1: Group A ON, Group B OFF (3 second hold)
    all_off()
    led1.value(1)
    led2.value(1)
    time.sleep(3)      # Group A holds for 3 seconds

    # Phase 2: Group B ON, Group A OFF (5 second hold)
    all_off()
    led3.value(1)
    led4.value(1)
    time.sleep(5)      # Group B holds for 5 seconds
```

Timing diagram:



19. Lab Programs — Series 2

Lab 2.1 — Binary Counter (Up/Down) with 4 LEDs

Objective: Display numbers 1–15 in binary on 4 LEDs, counting up then down. At maximum (15) and minimum (1), wait 2 seconds before reversing.

Wiring:

GP0 = LSB (bit 0, value 1)
GP1 = bit 1 (value 2)
GP2 = bit 2 (value 4)
GP3 = MSB (bit 3, value 8)

e.g., count=5 (0101): GP0=1, GP1=0, GP2=1, GP3=0

Program:

Lab 2.1: Binary Up/Down Counter using 4 LEDs
Count: 1→2→3→...→15 (wait 2s) →14→13→...→1 (wait 2s) →2→...

```
from machine import Pin
import time
```

```
# Define 4 LEDs mapped to bits 0–3
leds = [Pin(0, Pin.OUT), # bit 0 (LSB)
        Pin(1, Pin.OUT), # bit 1
        Pin(2, Pin.OUT), # bit 2
        Pin(3, Pin.OUT)] # bit 3 (MSB)
```

```
DELAY_NORMAL = 1 # 1 second between counts
DELAY_MINMAX = 2 # 2 second hold at min/max
```

```
def display_binary(value):
    """Output a 4-bit value to the 4 LEDs."""
    for bit in range(4):
        leds[bit].value((value >> bit) & 1) # Extract each bit
    print(f"Count: {value:2d} Binary: {value:04b}") # Debug output
```

```
# Start counting up from 1
```

```
direction = 1    # +1 for up, -1 for down
count = 1
```

```
while True:
    display_binary(count)

    # Check if at boundary
    if count == 15:
        time.sleep(DELAY_MINMAX) # Hold 2s at maximum
        direction = -1           # Switch to counting down
    elif count == 1 and direction == -1:
        time.sleep(DELAY_MINMAX) # Hold 2s at minimum
        direction = 1            # Switch to counting up
    else:
        time.sleep(DELAY_NORMAL) # Normal 1s delay

    count += direction           # Advance counter
```

Execution trace:

```
Count: 1 → 2 → 3 → ... → 14 → 15 [2s pause]
       → 14 → 13 → ... → 2 → 1 [2s pause]
       → 2 → 3 → ... → 15 [2s pause] → ...
```

Binary display examples:

Count	Bit3(GP3)	Bit2(GP2)	Bit1(GP1)	Bit0(GP0)
1	0	0	0	1
5	0	1	0	1
10	1	0	1	0
15	1	1	1	1

Key technique — bit extraction:

```
(value >> bit) & 1
# value >> bit : right-shift value by `bit` positions
# & 1          : mask to get only the lowest bit
# e.g., value=5 (0101), bit=2: (5>>2)=1, 1&1=1 → GP2=HIGH
```

Lab 2.2 — Odd/Even Binary Display with 4 LEDs

Objective: Display numbers cycling in a specific odd-then-even sequence in binary, counting up to 15 (max), hold 2s, then back to minimum, repeat.

Clarification of sequence from problem statement:

Example: 0001, 0010, 0011, ..., 1111, 1110, 1101, ..., 0001, 0010, 0011, ...

This is a standard up/down binary counter (1, 2, 3, ..., 15, 14, 13, ..., 1, 2, 3, ...) — the odd/even labelling refers to the values being displayed (they include both), but the problem says "display odd, even numbers" meaning both, sequentially.

Interpretation A — All numbers 1→15 up, then back down (same as 2.1 but without bi-directional reversal; instead wraps):

Per the example pattern 0001, 0010, 0011, ..., 1111, 1110, 1101, ..., 0001, this is count-up then count-down, wrapping.

Lab 2.2: Odd and Even numbers in binary using 4 LEDs
Sequence: 1,2,3,...,15 [wait 2s] 14,13,...,1 [wait 2s] 2,3,...

```
from machine import Pin
import time
```

```
leds = [Pin(0, Pin.OUT), # bit 0 (LSB)
        Pin(1, Pin.OUT), # bit 1
        Pin(2, Pin.OUT), # bit 2
        Pin(3, Pin.OUT)] # bit 3 (MSB)
```

```
DELAY = 1    # Normal delay between numbers
WAIT = 2     # Pause at max value
```

```
def display(value):
    """Show 4-bit binary on LEDs."""
    for bit in range(4):
        leds[bit].value((value >> bit) & 1)
    label = "ODD " if value % 2 != 0 else "EVEN"
    print(f'{label} | {value:2d} | {value:04b}')
```

```
while True:
    # Count UP: 1 to 15
    for n in range(1, 16):
        display(n)
```

```

if n == 15:
    time.sleep(WAIT) # Hold 2s at maximum (1111)
else:
    time.sleep(DELAY) # 1s delay

# Count DOWN: 14 to 1
for n in range(14, 0, -1):
    display(n)
    if n == 1:
        time.sleep(WAIT) # Hold 2s at minimum (0001)
    else:
        time.sleep(DELAY) # 1s delay

```

Interpretation B — Separate odd and even sequences:

If the intent is to show only odd numbers, then only even numbers:

Lab 2.2 (Alternate): Show odd numbers 1,3,5,...,15 then even 2,4,...,14
from machine import Pin
import time

```

leds = [Pin(0, Pin.OUT), Pin(1, Pin.OUT),
        Pin(2, Pin.OUT), Pin(3, Pin.OUT)]

```

```

def display(value):
    for bit in range(4):
        leds[bit].value((value >> bit) & 1)
    print(f"{'ODD ' if value%2 else 'EVEN'} {value:2d} = {value:04b}")

```

```

while True:
    # Odd numbers: 1, 3, 5, 7, 9, 11, 13, 15
    for n in range(1, 16, 2): # step=2, odd only
        display(n)
        time.sleep(1)
    time.sleep(2) # 2s at max (15)

    # Even numbers: 2, 4, 6, 8, 10, 12, 14
    for n in range(2, 15, 2): # step=2, even only
        display(n)
        time.sleep(1)
    time.sleep(2) # 2s at max (14)

```

Per the exact problem example pattern, use the first interpretation (sequential 1–15 up, then 14–1 down):

Complete annotated version:

```
# Lab 2.2 — FINAL: Binary up/down counter showing odd and even
# Pattern: 0001→0010→0011→...→1111 [2s]→1110→1101→...→0001 [2s]→repeat
```

```
from machine import Pin
import time
```

```
# GPIO pin to LED mapping (bit 0 = GP0 = LSB)
led_pins = [0, 1, 2, 3]
leds = [Pin(p, Pin.OUT) for p in led_pins]
```

```
def show(n):
    """
    Display integer n as 4-bit binary on 4 LEDs.
    Bit 0 (LSB) → GP0, Bit 3 (MSB) → GP3
    """
    for i in range(4):
        leds[i].value((n >> i) & 1)
    parity = "ODD " if n & 1 else "EVEN"
    print(f"{parity} | {n:2d} | GP3:{(n>>3)&1} GP2:{(n>>2)&1} "
          f"GP1:{(n>>1)&1} GP0:{n&1}")
```

```
while True:
    # — UP SWEEP: 1, 2, 3, ..., 15 —————
    for count in range(1, 16):      # 1 to 15 inclusive
        show(count)
        if count == 15:
            time.sleep(2)           # Hold 2 seconds at max (1111)
        else:
            time.sleep(1)           # 1 second between steps

    # — DOWN SWEEP: 14, 13, 12, ..., 1 —————
    for count in range(14, 0, -1):  # 14 down to 1 inclusive
        show(count)
        if count == 1:
            time.sleep(2)           # Hold 2 seconds at min (0001)
        else:
            time.sleep(1)
```

Sample Serial Output:

ODD | 1 | GP3:0 GP2:0 GP1:0 GP0:1
 EVEN | 2 | GP3:0 GP2:0 GP1:1 GP0:0
 ODD | 3 | GP3:0 GP2:0 GP1:1 GP0:1
 ...
 ODD | 15 | GP3:1 GP2:1 GP1:1 GP0:1 ← 2s pause here
 EVEN | 14 | GP3:1 GP2:1 GP1:1 GP0:0
 ODD | 13 | GP3:1 GP2:1 GP1:0 GP0:1
 ...
 ODD | 1 | GP3:0 GP2:0 GP1:0 GP0:1 ← 2s pause here
 EVEN | 2 | GP3:0 GP2:0 GP1:1 GP0:0
 ...

20. Quick-Reference Comparison Tables

20.1 μ P vs μ C — Exam Cheat Sheet

Feature	Microprocessor	Microcontroller
Memory	External	On-chip
Peripherals	External chips	Integrated
Chip count	Many	One
Power	High	Low
Cost (system)	High	Low
Complexity	High	Low
Use	Computing	Embedded control
Example	Intel i7 (423 pins)	ATmega328 (32 pins), RP2040 (56 pins)

20.2 RP2040 At-a-Glance

Parameter	Value
Cores	2 × ARM Cortex-M0+
Max Clock	133 MHz
On-chip RAM	264 kB SRAM

External Flash	Up to 16 MB (QSPI/XIP)
On-chip Flash	None (digit = 0)
GPIO	30 pins (GP0–GP29)
ADC	4 × 12-bit channels
UART	2
SPI	2
I ² C	2
PWM channels	16
PIO state machines	8 (2 blocks × 4)
USB	1.1 host+device
Package	QFN-56, 7×7 mm
Voltage (I/O)	1.8–3.3 V
Temperature	–20 °C to +85 °C
ISA	RISC
Flash type	NOR Flash (external)
Production guaranteed	Until Jan 2041

20.3 Pipeline Stages Reference

Stage	Abbr.	Key Hardware Used
Instruction Fetch	IF	PC, MAR, MBR, IR, Program Memory
Instruction Decode	ID	Instruction decoder, Register file read ports
Execute	EX	ALU, Barrel shifter
Memory Access	MA	MAR, MBR, Data memory
Write Back	WB	Register file write port

20.4 MicroPython Pin API Quick Reference

Statement	Effect
<code>Pin(n, Pin.OUT)</code>	Configure GPIO n as output
<code>Pin(n, Pin.IN)</code>	Configure GPIO n as input
<code>pin.value(1)</code>	Output HIGH (3.3 V)
<code>pin.value(0)</code>	Output LOW (0 V)
<code>pin.toggle()</code>	Invert output
<code>pin.value()</code>	Read current output/input value
<code>time.sleep(s)</code>	Delay s seconds
<code>time.sleep_ms(m s)</code>	Delay ms milliseconds
<code>time.sleep_us(u s)</code>	Delay us microseconds
<code>print("text")</code>	Output to serial monitor

20.5 Binary Encoding Reference (4-bit)

Decimal	GP3	GP2	GP1	GP0	Parity
1	0	0	0	1	ODD
2	0	0	1	0	EVEN
3	0	0	1	1	ODD
4	0	1	0	0	EVEN
5	0	1	0	1	ODD
6	0	1	1	0	EVEN
7	0	1	1	1	ODD
8	1	0	0	0	EVEN
9	1	0	0	1	ODD

10	1	0	1	0	EVEN
11	1	0	1	1	ODD
12	1	1	0	0	EVEN
13	1	1	0	1	ODD
14	1	1	1	0	EVEN
15	1	1	1	1	ODD

*End of ESM Unit 2 Master Notes — CS1120, RV University Coverage: All slides from Unit 2
PDF + Lab Programs 1.1, 1.2, 1.3, 2.1, 2.2*